

Хронограф — сопроводительная документация

Задача №7 «Автоматизированный сервис поиска нарушений на строительных площадках Москвы». Команда CyberCrew. Репозиторий: <https://github.com/acmtco/ISMC>

Документ составлен по разделу 5 технического задания.

1. Модель данных

Хранение разделено на два уровня: **потокковые файлы** (append-only, по камере) и **реляционная база** (агрегаты и то, что показывается пользователю). Причина разделения — объём: детекции по кадрам растут линейно от числа камер и суток, а пользователю нужны суточные агрегаты.

1.1 Справочники и потокковые файлы

Файл	Что хранит	Формат
<code>data/ref/objects.json</code>	Паспорт объекта: камеры, источники, полигоны зон	JSON
<code>data/ref/work_signatures.json</code>	Ресурсные отпечатки: «этап работ → техника»	JSON
<code>data/ref/productivity.json</code>	Нормы выработки для вывода плановых машино-часов	JSON
<code>data/ref/machine_hour_cost.json</code>	Стоимость машино-часа по классам, ₽	JSON
<code>data/interim/detections/<камера>.jsonl</code>	Одна строка на кадр: боксы, классы, треки, состояния, качество	JSON Lines

Строка `detections.jsonl` — основной контракт слоя восприятия:

```
{"camera_id":"CAM-01","ts":"2026-09-20T08:30:00+03:00",
"frame_uri":"data/replay/CAM-01/20260920T083000.jpg",
"quality":{"blur":0.12,"night":false,"occlusion":0.03,"score":0.94},
"objects":[{"track_id":17,"cls":"excavator","conf":0.91,
  "bbox":[412,388,180,120],"zone_id":"Z-PIT","state":"active",
  "state_conf":0.83,
  "features":{"centroid_mad":4.7,"area_mad":311.0,
    "flow_mag":1.94,"hsv_var":480.0}}]}
```

1.2 Реляционная схема

SQLite в разработке, PostgreSQL в продуктивном профиле — переключается переменной `HG_DB_URL` без правки кода. Шесть таблиц:

objects — объект строительства. `object_id` (PK), `name`, `address`, `timezone`.

cameras — камера объекта. `camera_id` (PK), `object_id` (FK), `title`, `source_kind` (`rtsp` / `hls` / `folder` / `replay`), `source_uri`, `capture_interval_sec`.

zones — зона выполнения работ на кадре камеры. `zone_id` (PK), `camera_id` (FK), `title`, `polygon` (JSON — список вершин в пикселях кадра).

schedule_works — строка календарного графика. `work_id` (PK), `object_id` (FK), `wbs`, `name`, `work_type`, `zone_id`, `start_plan`, `finish_plan`, `volume_unit`, `volume_qty`, `planned_mh` (JSON — плановые машино-часы по классам техники), `predecessors` (JSON).

machine_hours — суточный агрегат, ядро системы. Ключ — (дата × зона × класс техники). `date`, `object_id` (FK), `zone_id`, `equipment_class`, `units_seen`, `mh_present`, `mh_active`, `mh_idle`, `utilization`, `coverage`, `confidence`.

deviations — выявленное отклонение, то, что видит руководитель. `deviation_id` (PK), `object_id` (FK), `detected_at`, `period_from`, `period_to`, `work_id`, `zone_id`, `type` (P0–P4), `severity`, `observed` (JSON — что наблюдали), `expected` (JSON — что ожидали), `spi`, `delay_days`, `idle_cost_rub`, `explanation` (формулировка на русском), `recommendation`, `evidence_frames` (JSON — ссылки на кадры-улики), `evidence_chart`, `evidence_confidence`.

Состав полей `deviations` продиктован требованием объяснимости: карточка обязана содержать числа плана, числа факта, кадры-улики, ссылку на строку графика и формулировку, понятную прорабу.

1.3 Как считаются машино-часы

Снимки приходят редко (раз в 10–20 минут), поэтому каждый кадр «отвечает» за интервал до следующего кадра, ограниченный сверху одним часом — иначе единственный ночной снимок приписал бы себе всю ночь.

Три правила исключения:

- состояние `parked` не учитывается вовсе — ни в присутствии, ни в работе;
 - состояние `unknown` не учитывается — если система не смогла определить состояние, честнее не засчитывать часы, чем угадывать;
 - кадр с качеством ниже 0.5 не участвует в расчёте, но попадает в `coverage` суток, чтобы было видно, что данных не хватило.
-

2. Организация кода

services/perception/	кадр → детекции → треки → состояние
classes.py	канонический перечень классов техники
source.py	источники кадров: RTSP, HLS, папка, replay
quality.py	оценка качества кадра
detect.py	обёртка YOLO11
track.py	BoT-SORT + склейка разрывов треков
zones.py	привязка объекта к полигону по точке опоры
state.py	классификатор «работает / простой / на приколе»
pipeline.py	сборка перечисленного в один проход
services/analytics/	детекции → машино-часы → отклонения → прогноз
machine_hours.py	суточная агрегация
schedule_io.py	импорт ГПП из XLSX и MS Project XML
matching.py	ресурсный отпечаток и match_score
deviations.py	правила P0–P4
forecast.py	SPI, прогноз срыва срока, обратная задача
economics.py	КИТ, потери от простоя
ru_text.py	русские формулировки и словоформы
services/api/	FastAPI: 12 эндпоинтов, планировщик, PDF-акты
web/	React SPA, пять экранов
scripts/	синтетика, обучение детектора, сквозная оценка
config/	все пороги и веса правил
docs/	контракты, методика, метрики
tests/	205 автотестов

Принцип, выдержанный по всему коду: ни одного магического числа. Все пороги вынесены в `config/*.yaml`, все справочные значения — в `data/ref/*.json`. Методику сопоставления можно править без программиста.

Файл конфигурации	Что задаёт
<code>config/perception.yaml</code>	Пути к весам, порог уверенности детектора, окно признаков, параметры склейки треков
<code>config/matching.yaml</code>	Веса формулы <code>match_score</code> , порог низкой уверенности
<code>config/thresholds.yaml</code>	Пороги правил P0–P4, границы светофора SPI, окно прогноза
<code>config/api.yaml</code>	Интервал опроса камер, время ночного пересчёта

Комментарии расставлены там, где логика нетривиальна: почему трекам нужна собственная склейка поверх BoT-SORT, почему зона определяется по нижней грани бокса, почему припаркованная техника исключается из машино-часов, почему у правил именно такой

приоритет. Каждый нетривиальный модуль начинается с docstring, объясняющего решение и его ограничения.

3. Условия и ограничения

Раздел написан намеренно подробно: понимание границ применимости важнее эффективных цифр.

3.1 На каких данных обучены модели

Детектор техники. Дообучается от предобученных весов YOLO11 на открытом размеченном наборе строительной техники (YOLO-формат) и на собственной разметке снимков, переданных организаторами. Скрипт обучения — `scripts/train_detector.py`, воспроизводится одной командой. Состав обучающей выборки и достигнутые метрики фиксируются в `docs/04-metrics.md` только прогоном `scripts/eval_end2end.py`.

Ограничение: набор снимков от организаторов (100 изображений) приходит **без разметки**, отдельный проверочный набор не предоставляется. Проверочная выборка формируется самостоятельно с учётом различий в освещённости, сезонности и ракурсах.

Классификатор состояния. Обучен на признаках движения, посчитанных по синтетическому бенчмарку с честным ground truth. Признаки: медианное абсолютное отклонение центроида и площади бокса, средняя величина оптического потока Фарнебака внутри бокса, дисперсия HSV внутри бокса.

Ограничение, зафиксированное честно: на отложенной выборке из тех же синтетических распределений модель даёт высокую точность, но на реальных пикселях качество заметно ниже — спрайты синтетики статичны, их текстура не коррелирует с состоянием так, как у настоящей техники. Это означает, что правила P1 и P2, опирающиеся на долю активного времени, пока работают хуже правил P3 и P4, опирающихся на сам факт присутствия и состав техники. Числа приведены в `docs/04-metrics.md`.

3.2 Какие типы техники распознаются

Двенадцать классов. Восемь из технического задания:

Класс в системе	Русское название
excavator	экскаватор
dump_truck	самосвал
roller	каток
knuckle_boom_crane	кран-манипулятор
concrete_mixer	бетономеситель
bulldozer	бульдозер
truck	грузовик
mobile_crane	автокран

Четыре добавлены сверх перечня (техническое задание это прямо разрешает), поскольку они требуются ресурсным отпечаткам работ из справочника ДГП: `tower_crane` (башенный кран), `drilling_rig` (буровая установка), `concrete_pump` (бетононасос), `loader` (погрузчик).

Различие `truck` и `dump_truck` содержательное: самосвал возит грунт, бортовой грузовик — материалы, для ресурсного отпечатка это разные ресурсы.

Люди не распознаются сознательно — контроль охраны труда относится к другой задаче конкурса. Госномера техники не читаются.

3.3 Для каких этапов работ настроена логика сопоставления

Ресурсные отпечатки заданы для семи обобщённых видов работ: разработка котлована, свайные работы, фундаментная плита, монтаж каркаса, обратная засыпка, инженерные сети, фасадные работы.

Отдельно оговорено ограничение по фасадным работам: ключевая техника для них — строительный подъёмник, класса для него в перечне нет, поэтому машино-часами этот вид работ система не измеряет и отклонений по нему не формирует. Это записано прямо в справочнике, а не умалчивается.

Официальный «Сводный перечень строительных работ» ДГП содержит 380 позиций. Текущие семь типов — обобщение; сопоставление с полным справочником запланировано до финальной сдачи.

3.4 Прочие ограничения

- Перевод пикселей в метры не реализован: поле `homography` в паспорте камеры зарезервировано, но не используется. Площади зон задаются вручную.
- Ритм работы (число рейсов техники за смену) в движке правил принимается нейтральным: подсчёт реальных рейсов требует данных о входе и выходе из зоны по трекам, которых нет в текущем контракте агрегата.

- Система не претендует на юридически значимый учёт — она даёт основание для проверки, а не заменяет её.
- Стоимость машино-часа и нормы выработки — оценочные, требуют сверки с проверяемым источником (ЕНиР/ГЭСН) перед использованием в расчётах экономического эффекта. Это помечено внутри самих справочных файлов.

4. Сборка, установка и запуск

4.1 Требования

- Python 3.11
- Node.js 20 или новее (для интерфейса)
- Docker и Docker Compose (необязательно, для запуска одной командой)

Камера, GPU и доступ в интернет для запуска **не требуются**: система работает в режиме `replay` на предзаписанных снимках.

4.2 Запуск через Docker Compose

```
git clone https://github.com/acmtco/ISMIC.git
cd ISMC
cp .env.example .env
docker compose --profile demo up
```

Интерфейс — <http://localhost:5173>, API и документация — <http://localhost:8000/docs>

4.3 Запуск без Docker

Backend:

```
python3.11 -m venv .venv
.venv/bin/pip install -e ".[dev]"
.venv/bin/uvicorn services.api.main:app --port 8000
```

Frontend, в отдельном терминале:

```
cd web && npm install && npm run dev
```

4.4 Проверка работоспособности

```
.venv/bin/python -m pytest -q && .venv/bin/ruff check .
```

Ожидаемый результат — 205 пройденных тестов без замечаний линтера. Тесты не требуют сети, GPU и камеры.

4.5 Генерация синтетического бенчмарка

```
.venv/bin/python -m scripts.make_synthetic --days 30
```

Создаёт 30 суток снимков с внедрёнными отклонениями, файл достоверной разметки и перечень ожидаемых отклонений — по ним считаются метрики.

4.6 Обучение детектора

```
.venv/bin/python -m scripts.train_detector \  
  --yolo-dir <папка с размеченным набором> \  
  --class-map "0=excavator" \  
  --epochs 60 --device mps
```

`--device` принимает `mps` (Apple Silicon), `cpu` или номер GPU. Результат сохраняется в `models/yolo11n_construction.pt` — путь, который ожидает `config/perception.yaml`.

4.7 Сквозная оценка качества

```
.venv/bin/python -m scripts.eval_end2end
```

Прогоняет весь конвейер по бенчмарку и заполняет `docs/04-metrics.md` таблицами и графиками. **Это единственный источник числовых показателей в проекте** — значения, полученные иначе, в документы и презентацию не попадают.

4.8 Переменные окружения

Переменная	По умолчанию	Назначение
HG_MODE	replay	live включает опрос камер по расписанию
HG_DATA_DIR	./data	Корень данных
HG_CONFIG_DIR	./config	Корень конфигурации
HG_DB_URL	sqlite:///./data/hronograf.db	Хранилище, сюда же PostgreSQL
HG_API_PORT	8000	Порт API
HG_WEB_PORT	5173	Порт интерфейса
HG_CORS_ORIGINS	http://localhost:5173	Разрешённые источники запросов
HG_REPLAY_SPEED	60	Ускорение «машины времени»
HG_LOG_LEVEL	INFO	Уровень логирования